

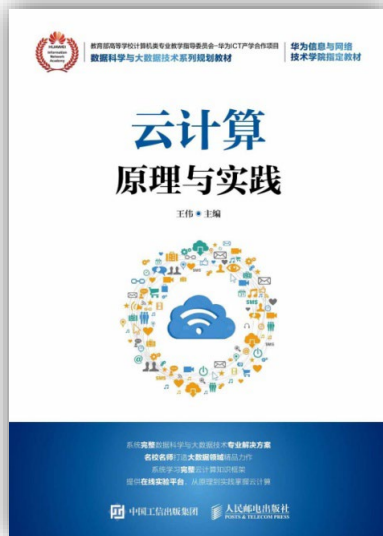


# 云计算原理与实践

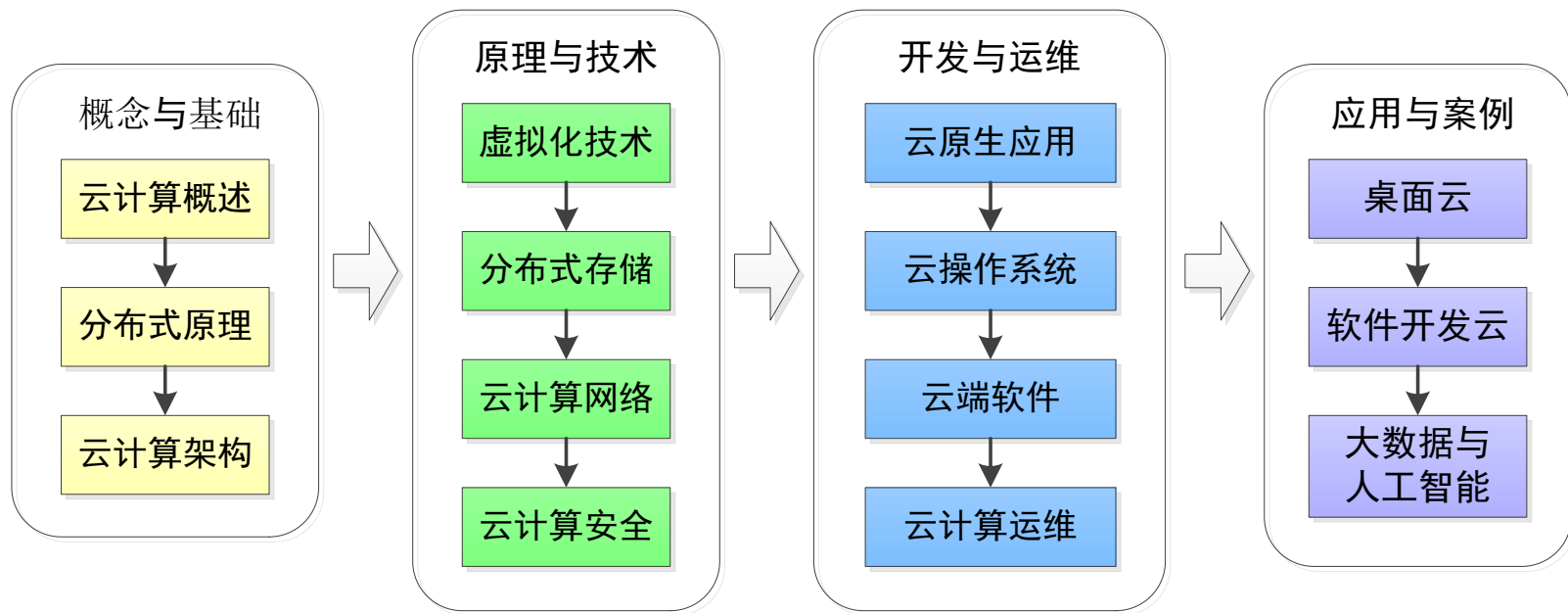
Principles and Practice of Cloud Computing

# 云计算原理与实践

## Principles and Practice of Cloud Computing



# 《云计算原理与实践》课程总览



# Outline

- 3.1 云计算的本质
- 3.2 云计算的架构
- 3.3 云栈和云体
- 3.4 软件定义的数据中心
- 3.5 实践：OpenStack

## 3.1 云计算的本质

3.1.1 革命性概念：IT作为服务

3.1.2 云计算系统工程

3.1.3 云数据中心

3.1.4 云的工作负载模式

3.1.5 云计算的规模效应

## 3.1.1 革命性概念：IT作为服务

- 云计算将所有IT资源包装为服务予以销售，也就是所谓的“IT作为服务”。绝不可以轻看IT作为服务这个概念。尽管在主机时代就是如此，但IT作为服务这种理念仍然具有颠覆性的特点。因为我们大部分人已经习惯拥有自己的IT资产，对IT资产由别人拥有这种模式抱有潜意识的抵触情绪。不过，如果仔细分析这个问题，我们就会发现，IT作为服务是顺理成章的一种自然演变。

## 3.1.2 云计算系统工程

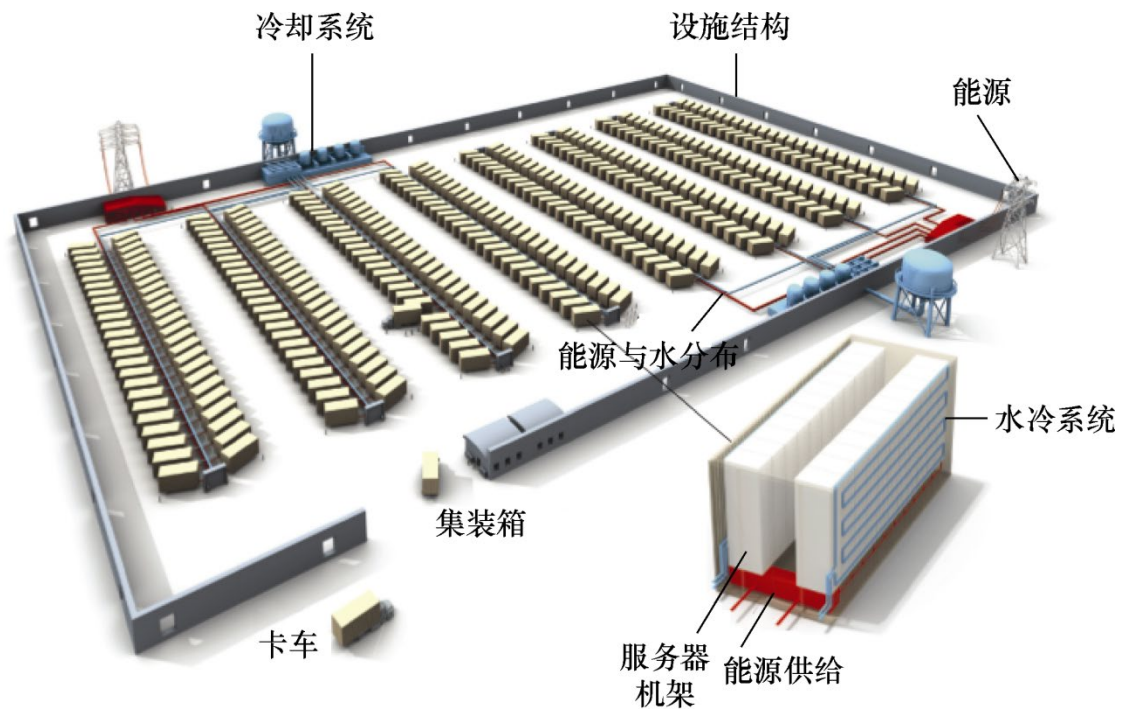
- 好的组织需要优秀的管理者，而云计算将大量计算资源组织在一起，共同工作，那么云计算需要给出一种针对大规模系统的科学管理办法。这种方法能够解决资源组织管理过程中的各种问题。例如：在增加节点、扩大系统规模的同时，还能保证系统性能的近线性提高。在系统可能出问题的情况下，保证系统整体的稳定运行。在面临不同的业务需求时，快速重新组织资源，以新的架构适应变化。这些都要求云计算创新性地将各种技术组织起来，“调和”实现各种功能，即所谓的“**系统工程**”。

## 3.1.3 云数据中心

目前，云数据中心的构造主要有两种模式：

- 一种是**传统模式**，即建机房、布线、放置机器，然后连接起来。
- 还有一种数据中心是**基于集装箱的数据中心**。这种模式由谷歌公司首创，使用集装箱作为机房，每个集装箱里安置有上千台服务器，最多可达2500台，集装箱可以叠起或并排放置，集装箱之间通过线缆连接形成巨大的数据中心。

图3.1 一个典型的数据中心



## 3.1.4 云的工作负载模式

- 模式1：时开时停模式
- 模式2：用量迅速增长模式
- 模式3：瞬时暴涨模式
- 模式4：周期性增减模式

## 3.1.5 云计算的规模效应

- 云计算的规模效应让用户使用IT资源的门槛大为降低。用户无须制定长期投资。如果有更多的客户，则购买更多的处理能力和存储容量，支付更多的费用；如果业务有所下降，则购买较少的处理能力和存储容量，这样支付的费用将降低。云计算平台可以真正达到24小时/7天×/365天运转。

## 3.2 云计算的架构

3.2.1 计算架构的进化

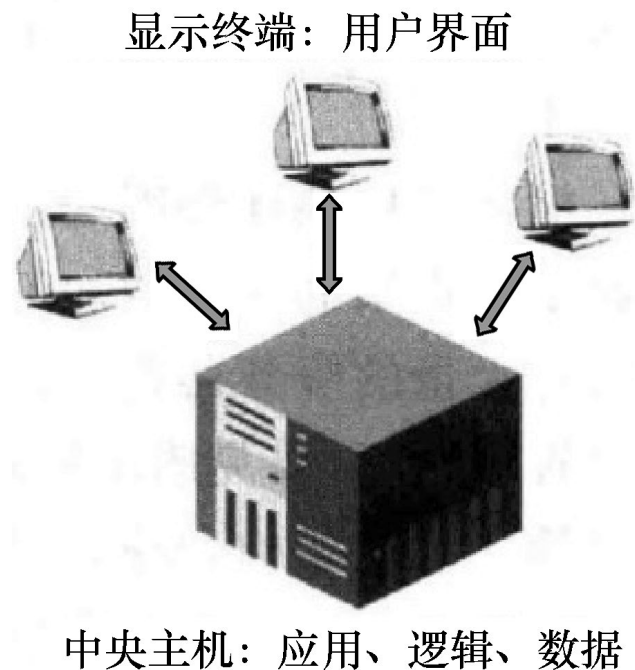
3.2.2 一般云计算架构的二维视角

## 3.2.1 计算架构的进化

- 计算机出现后，计算机的软硬件都经历了长时间的演变，其中计算范式过从中央集权计算（主机计算）到客户机服务器计算，再到浏览器服务器计算，再到混合计算模式。不同的计算范式对应的是不同的计算架构，而每一种计算架构都与其所在的历史时期相符合。

# 1. 中央集权架构

- **中央集权架构**对应的是中央集权计算范式。在这种架构下，所有的计算及计算资源、业务逻辑都集中于一台大型机或者主机，用户使用一台仅有输入和输出能力的显示终端与主机连接来进行交互。



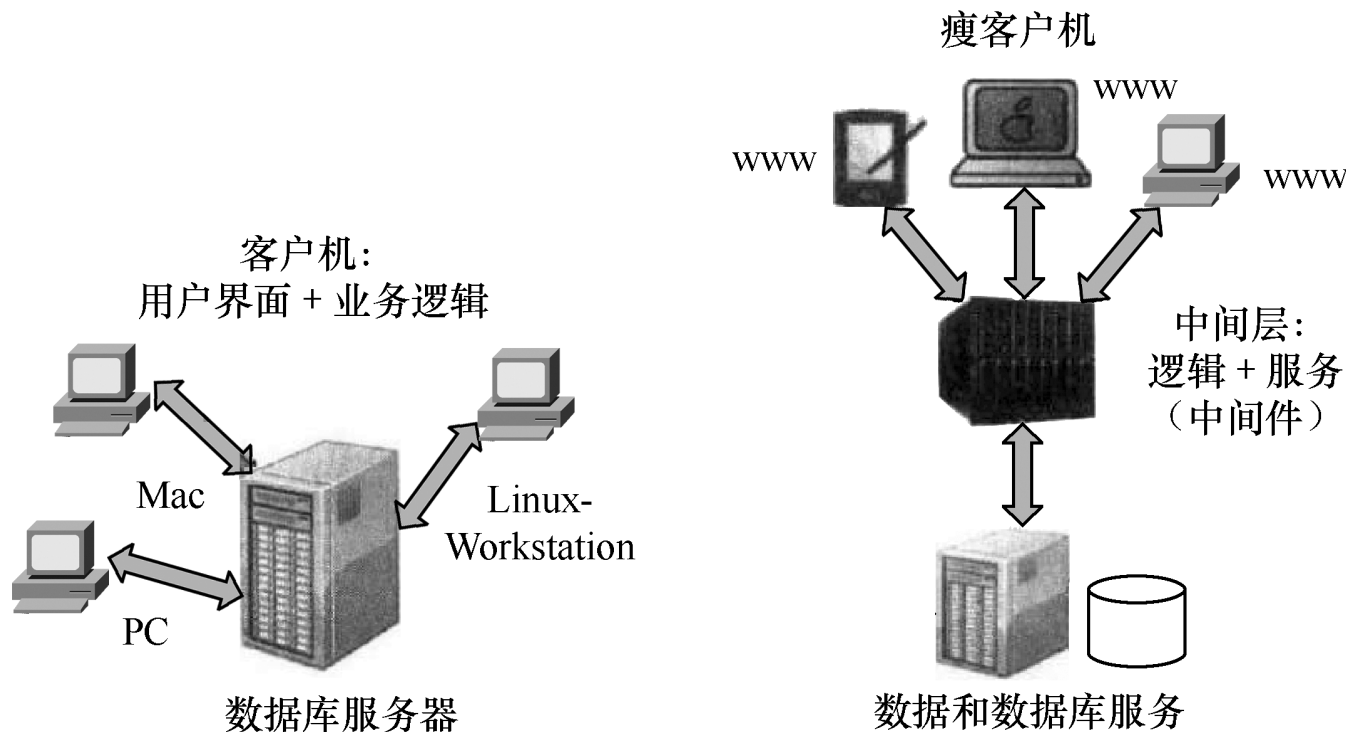
## 2. 客户机/服务器（C/S）架构

- **客户机/服务器（C/S）架构**对应的是同名计算范式。计算任务从单一主机部分迁移到客户端。客户端承载少量的计算任务和所有的I/O任务，服务器承载主要的计算任务。客户机在执行任务前先与主机进行连接，并在活跃的整个期间内保持与主机的持续连接。通常情况下，客户机通过远程过程调用来使用服务器上的功能和服务。

### 3. 中间层架构

- **中间层架构**对应的是多层客户机/服务器计算范式。它是在对客户机/服务器架构改进而产生的，其目的是简化和提升伸缩能力。所采用的方法是将业务逻辑和数据服务分别放在两个服务器上，客户机与中间服务器连接，中间层与数据服务层连接，客户机对数据的访问由中间层代理完成。图 3.10所示是中间层架构的示意图。

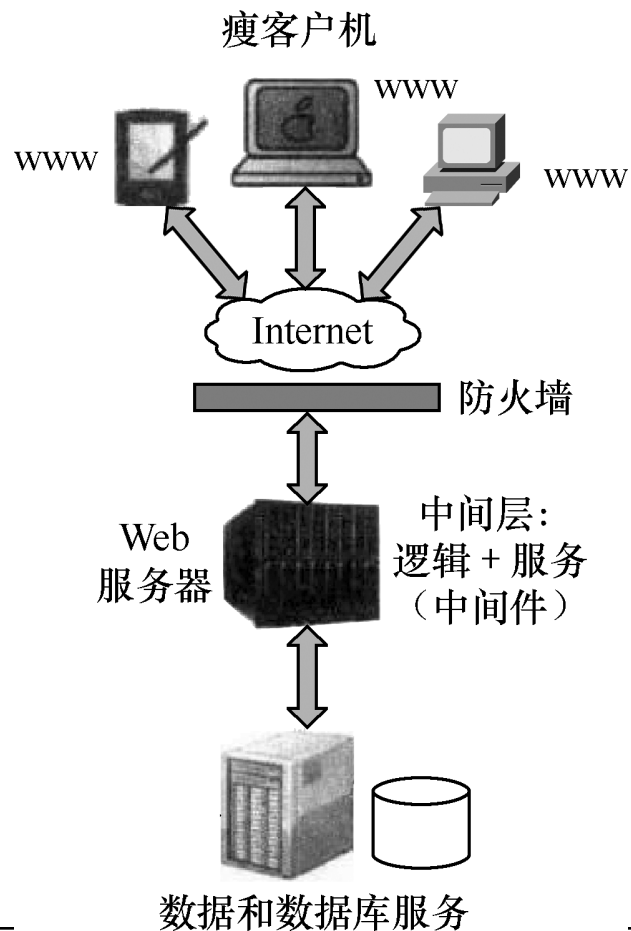
# 图3.10 中间层计算架构



## 4. 浏览器/服务器（B/S）架构

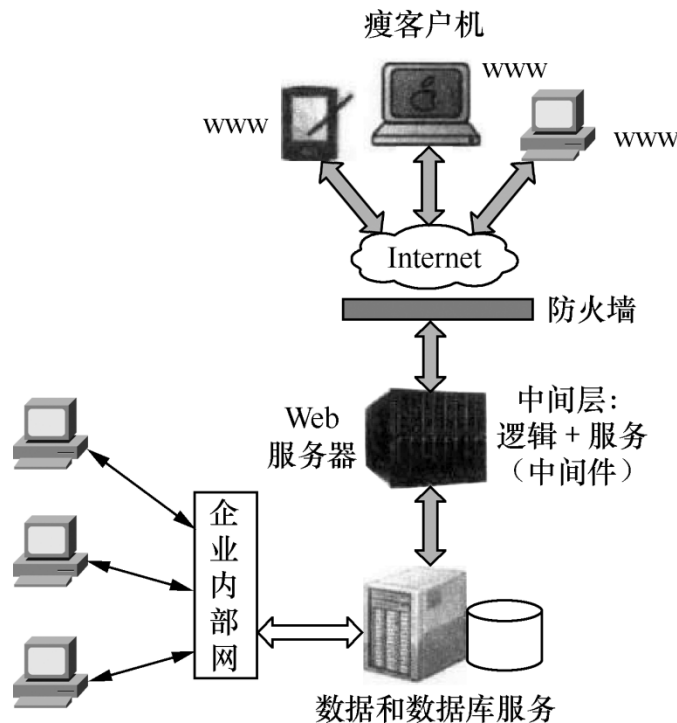
- **浏览器/服务器架构**对应的是浏览器/服务器计算范式。这种架构是对客户机和中间层的内涵进行改动后的中间层计算架构的扩展。对中间层的改动体现在中间层和客户机之间增加了一层Web服务器层，Web服务器可以将中间件的各种差异屏蔽掉，提供一种通用的用户访问界面。对客户机的改动则体现在负载的进一步缩减，从承载部分计算任务改变为只显示和运行一些基于浏览器的脚本程序的状态。

图3.11 浏览器/服务器计算架构



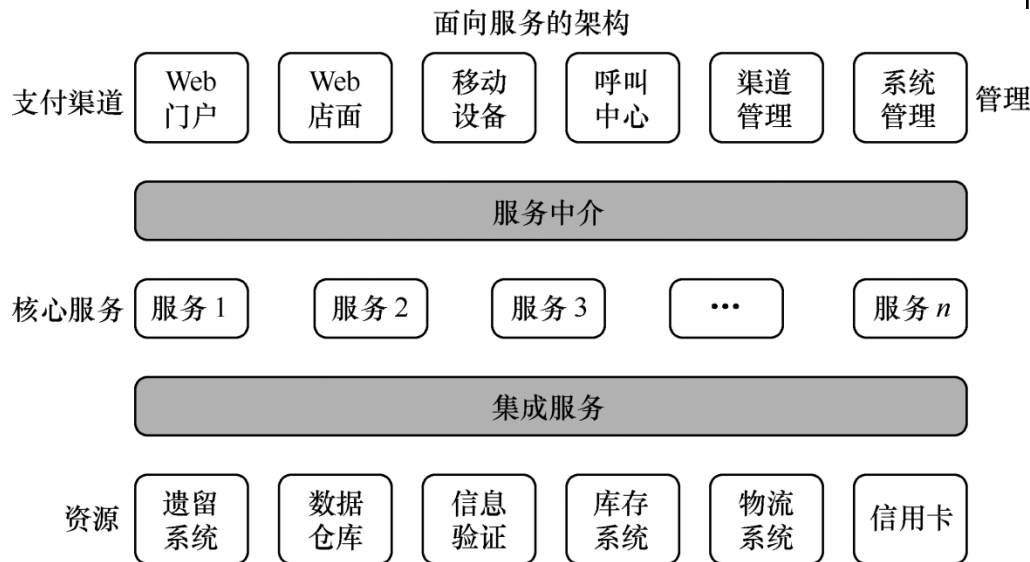
## 5. C/S与B/S混合架构

- **C/S与B/S混合架构**对应的是混合计算范式。在应用的发展中，没有一种计算范式适合所有的场景，没有一种计算架构适合所有的应用。故而衍生出了C/S与B/S混合架构，即客户机服务器和浏览器服务器两种架构并存的一种计算架构。



## 6. 面向服务的架构

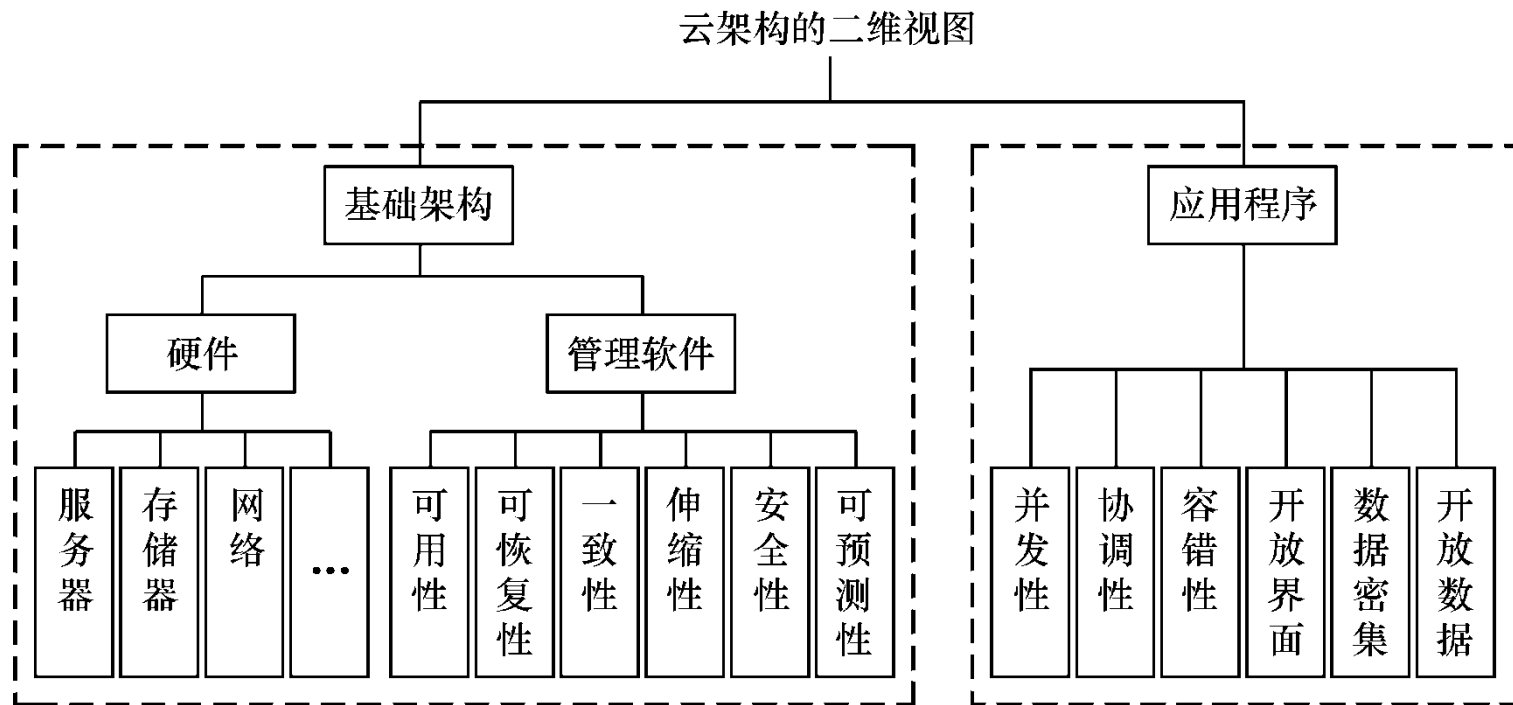
- 在面向服务的计算架构下，每个程序做本职工作，同时将服务暴露出来提供给其他程序使用，多个程序通过一个统一的（服务请求）界面协调工作。相对于单一系统来说，此种系统能够将复杂性限制在可控范围内，从而让整个系统的管理更加容易。



## 3.2.2 一般云计算架构的二维视角

- 从不同的角度来看，云计算架构的复杂性有一定的差异性。在最易于理解的二维视角下，云计算架构由两个部分组成：前端和后端。前端是呈现给客户或计算机用户的部分，包括客户的计算机网络和用户用来访问云应用程序的界面如Web浏览器；后端则是我们常说的“云”由各种组件（如服务器、数据存储设备、云管理软件等）构成。

图3.14 云计算架构的二维示意图



# 1. 基础架构的分层结构



图3.15 云基础架构的分层结构

## 2. REST架构：云计算的软件架构

REST体现了Web架构的特征：源服务、网关、代理和客户。其最大的特点是除了参与者的行为规范，对其中的个体组件没有任何限制。

REST架构的主要特点如下。

- 组件交互的伸缩性：参与交互的组件数量可以无限扩展。
- 界面的普遍性：IT界人士都熟悉REST的界面风格。
- 组件发布的独立性：组件可以独立发布，无须与任何组件进行事先沟通。
- 客户机/服务器模型：使用统一的界面来分离客户机和服务器。
- 无状态连接：客户机上下文不保存在服务器中，每次请求都需要提供完整的状态。

### 3. 云应用程序的结构

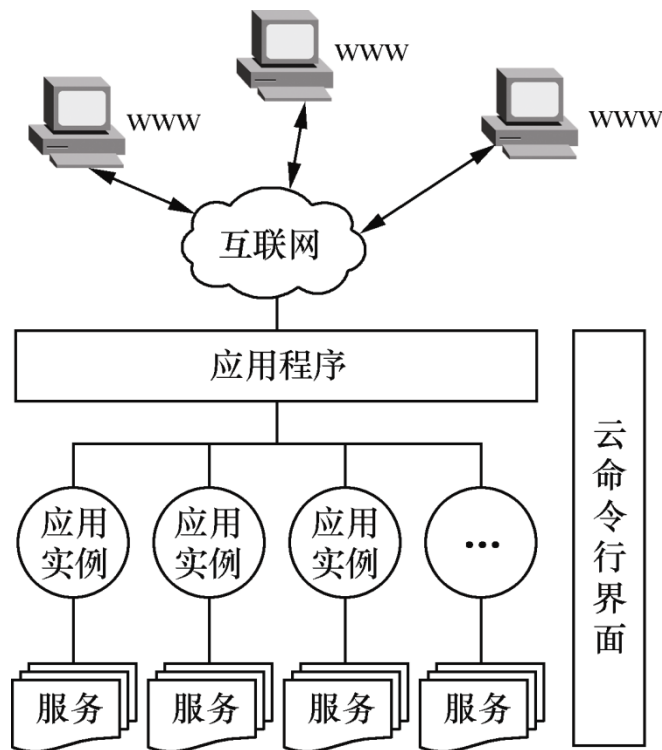


图3.16 云应用程序的软件结构

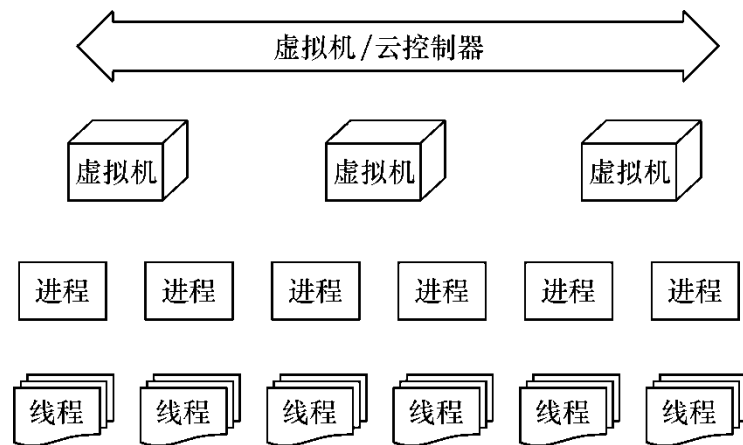


图3.18 将云平台看作应用所展示出来的架构

## 3.3 云栈和云体

3.3.1 云栈

3.3.2 云体

3.3.3 一切皆和服务

## 3.3.1 云栈

- **云栈**又称云平台，是在云上面建造的运行环境。它能够支持应用程序的发布、运行、监控、调度、伸缩，并为应用程序提供辅助服务的机制，如访问控制和权限管理等。如微软的Windows Azure、谷歌的App Engine、VMWare的Cloud Foundry都是云平台。
- 在云栈里，每一层都提供一种抽象。最下面的是物理硬件层，之后每往上一层，其离物理现实的距离就更远一些，易用性就会增加一分。每一层用来实现抽象的手段都是某种或某几种服务，也称为功能。如果两个服务处于等价的抽象层，则属于云栈里的同一层。

图3.19 云计算的纵向云栈架构（左）和传统计算机系统结构（右）



# 1. 三层云栈架构

- 在三层模式下，云计算可以很简要地概括为IaaS、PaaS、SaaS，也就是基础设施即服务、平台服务、软件服务。其中基础设施即服务可以称为效用计算（Utility Computing），平台即服务可称为弹性计算（Elastic Computing），软件即服务可称为按需应用（On-demand Applications）。

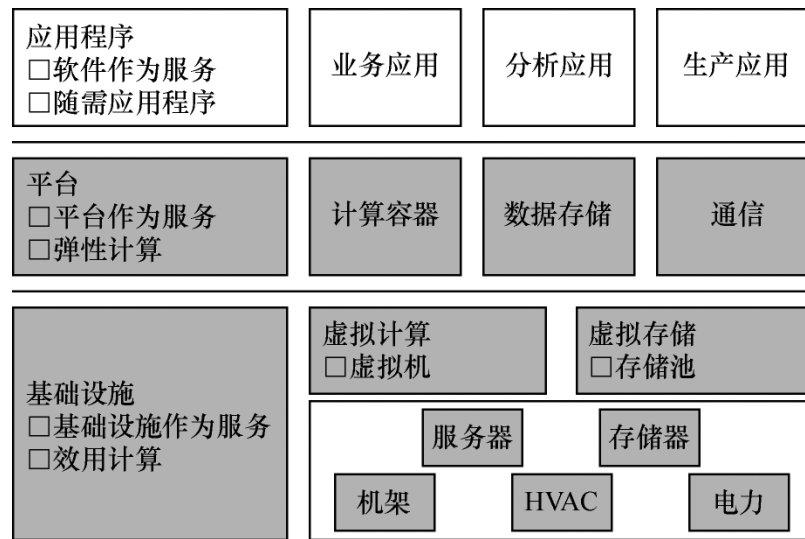


图3. 20 云栈的三层模式

# 三层模型里面的每一层的能力和特点

- (1) 基础设施即服务层
- (2) 平台即服务层
- (3) 软件即服务层

SaaS的主要特点有如下几点：

- 基于网络（一般为Web模式）进行远程访问的商用软件；
- 集中式管理，而非分散在每个用户站点；
- 应用交付一般接近一对多模型，即所谓的单个实例多个租户架构；
- 按照用量计费（实际中一般按月或其他时间周期进行计费）。

## 2. 四层云栈架构

- **四层云栈模式**是三层云栈模式的一个简单变种。这种模式将三层模式里的基础设施层进行了分解，将里面的硬件部分和虚拟化部分分离成为两层：一层为硬件层，一层为虚拟资源层，如图3.24所示。

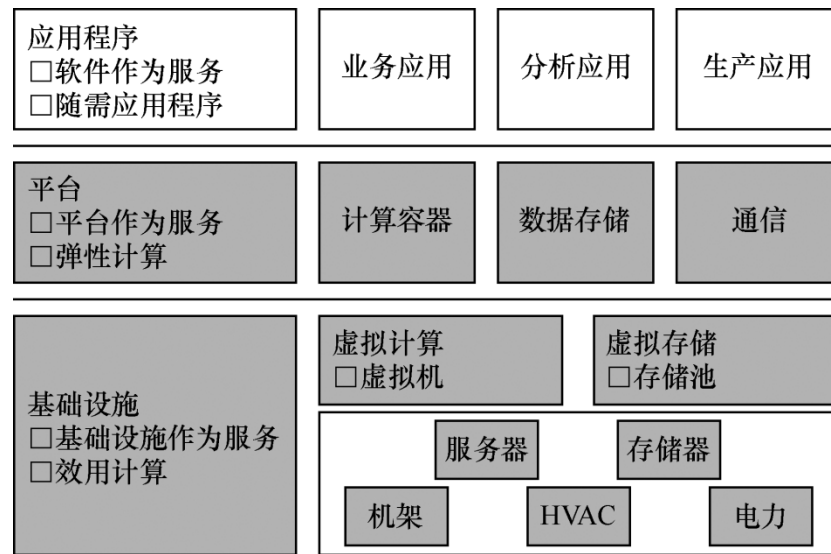


图3. 24 四层云栈模型

### 3. 五层云栈模型

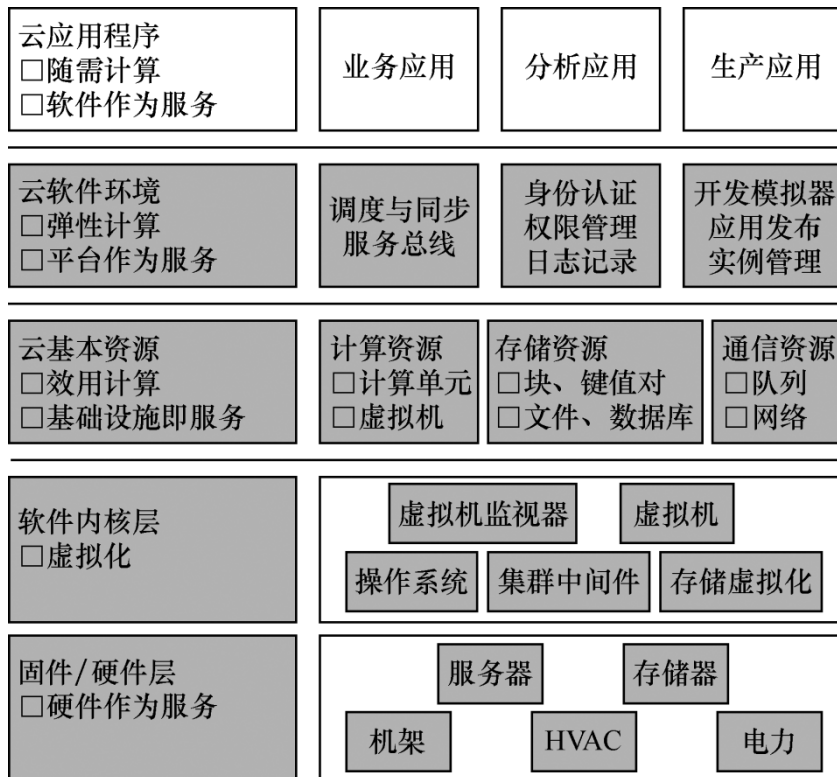
第一层：固件/硬件层

第二层：软件内核层

第三层：云基本资源层

第四层：云软件环境层

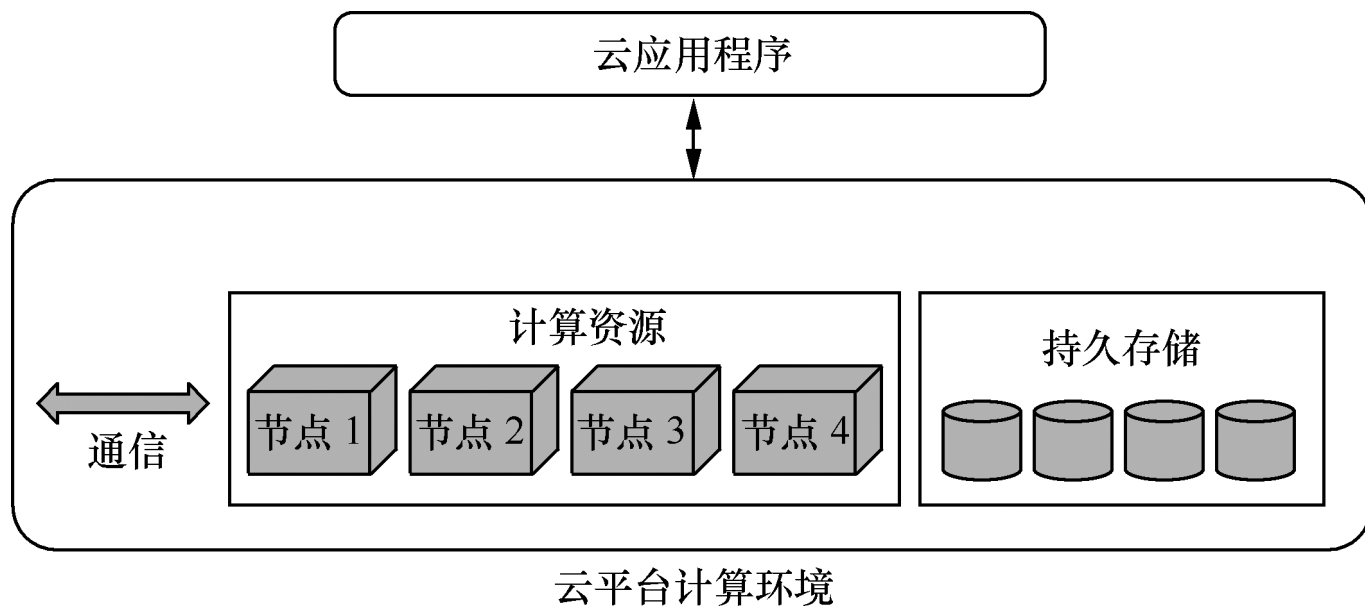
第五层：云应用程序层



## 3.3.2云体

- **云体**是云计算的物质基础，是云计算所用到的资源集合。它是构成云计算的软硬件环境，如网络、服务器、存储器、交换机等，通过网络连接在一起。某些情况下，广义的云体也可以包括数据中心及其辅助设施如电力、空调、机架、冷却等系统。鉴于当前的云计算都是基于数据中心来进行，云体就是数据中心。

# 图3.27 云平台的计算环境



### 3.3.3 一切皆和服务

- 无论是横向云体架构还是纵向云栈架构；无论是三层结构、四层结构，还是五层结构；无论是公有云、私有云，还是混合云或其他云；更无论是用量暴增、是周期性增减，还是用量稳定增长，都不能改变云计算的本质——服务。如果用一个短语来描述云计算，那就是：**IT即服务**。
- 云计算的本质就是IT作为服务涵盖了基础设施即服务、平台即服务、软件即服务或任何X即服务。在这种情况下，用户原来需要承担的IT资产采购、配置、运维的责任几乎全部转移到了各种服务供应商身上，从而可以轻装上阵，专注于自己的核心业务，不用为自己并不擅长的后勤花费巨大的人力和物力。

## 3.4 软件定义的数据中心

3.4.1 数据中心的历史

3.4.2 继续发展的推动力

3.4.3 软件定义的必要性

3.4.4 软件定义数据中心的架构分析

3.4.5 软件定义数据中心的发展

## 3.4.1 数据中心的历史

- **数据中心**（Data Center）是数据集中存储、计算、交换的中心。从硬件角度考虑，它给人最直观的印象就是计算设备运作的环境。故而数据中心的发展是与计算机（包括分化出的存储和网络设备）的发展紧密联系在一起。

## 3.4.1 数据中心的历史

- 20世纪60年代，商用计算机得到大量应用，其中最具代表性的是IBM的主机（Mainframe）系列。这些都是重达几十吨、占地数百平方米的庞然大物，与之略显不相称的是这些机器缓慢的计算速度和较小的数据存储规模。在当时，拥有这样一台计算机代价很高，而一个机房同时部署几台就更是异想天开。
- 20世纪80年代，随着大规模集成电路的发展，出现了大量相对廉价的微型计算机。数据的存储和计算呈现一种分散的趋势，越来越多的微型计算机被部署在政府、公司、医院、学校等。信息的交换依靠磁盘、磁带等介质。到了90年代，计算的操作变得越来越复杂，原有的微型计算机开始扮演客户端的角色，而大型的任务如数据库查询被迁移到服务器端，著名的客户端/服务器模式开始大行其道，直接推动了数据中心的发展。

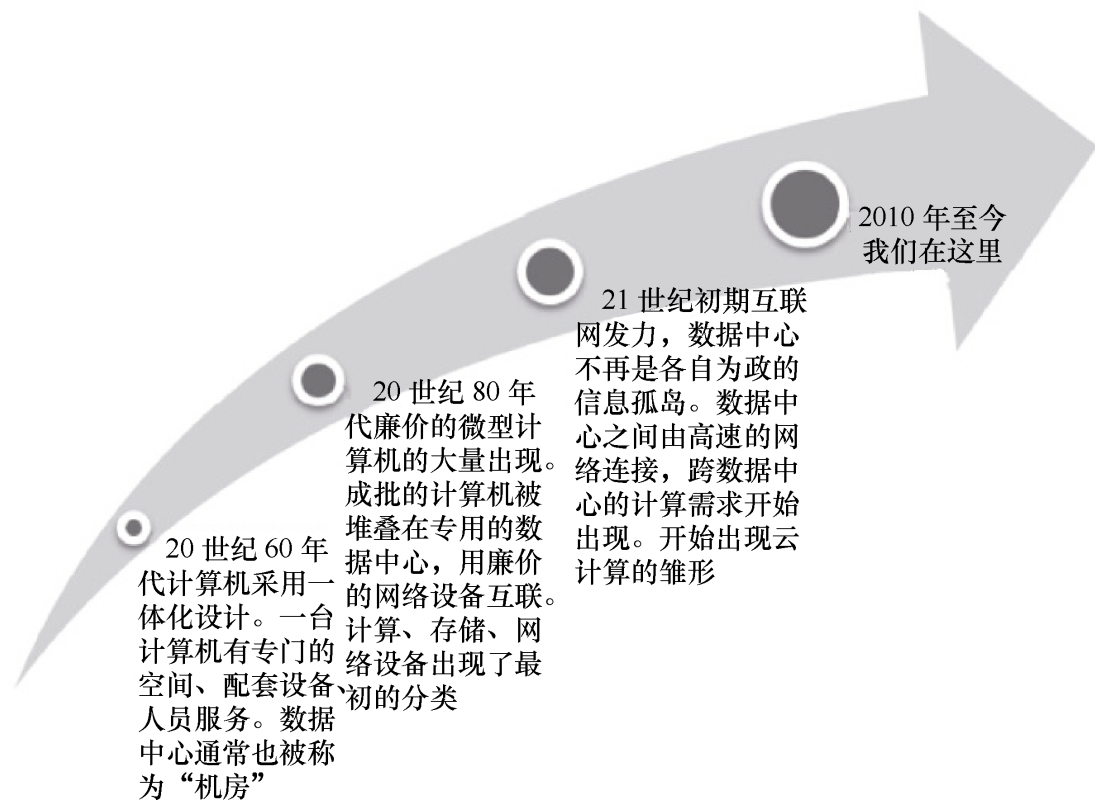
图3.28 IBM主机机房（左）和现代数据中心（右）



## 3.4.1 数据中心的历史

- 进入21世纪，互联网成为社会发展的主角，数据中心从技术发展到运行规模，都经历了前所未有的发展高潮。几乎所有的公司都需要高速的网络与Internet相连，公司的运营对于IT。IDC（Internet Data Center）就应运而生了，这是第一次出现以运营数据中心为主要业务的公司。由于竞争的需要，IDC竞相采用最新的计算机，采购最快速的网络连接设备和存储设备，应用最新的IT管理软件和管理流程，力图使自己的数据中心能吸引更多的互联网用户。

# 图3.29 数据中心的发展



## 3.4.2 继续发展的推动力

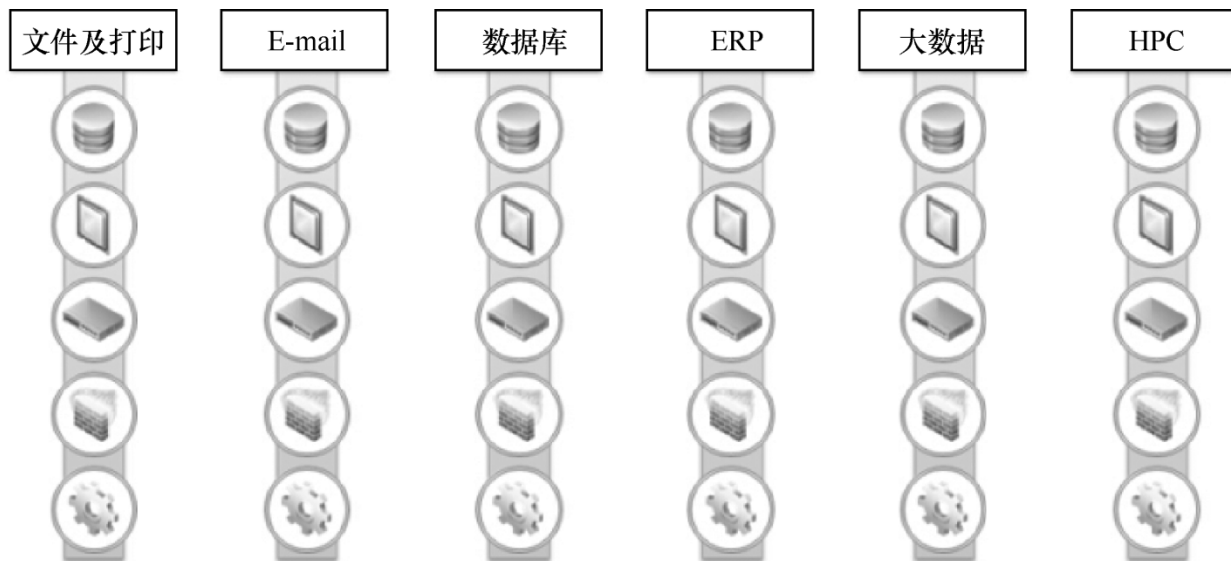
尽管数据中心发展完善，管理模式也很成熟，但对于数据中心系统管理员来说，传统模式的数据中心仍然存在着许多问题。

1. 过多的机器
2. 机器的利用率过低
3. 应用迁移太困难
4. 存储需求增长得太快

### 3.4.3软件定义的必要性

由于上述所说的困难，数据中心的管理员、应用系统的开发人员、最终用户，都认识到将数据中心的各个组成部分从硬件中抽象出来、集中协调与管理、统一提供服务是一件很重要的事情。如图3.30所示，在传统的数据中心中，如果需要部署一套业务系统，例如文件及打印服务，就要为该业务划分存储空间，分配运行文件及打印服务的服务器，配置好服务器与存储的网络。

# 图3.30 传统数据中心中的资源



## 3.4.4软件定义数据中心的架构分析

- 需求推动着软件定义数据中心的发展。回顾之前描述的发展路径，可以从系统分析的角度，描述出软件定义数据中心包括哪些部分或层次，以及实现这些组件需要的关键技术和整个系统提供的交互接口。

# 1. 层次细分

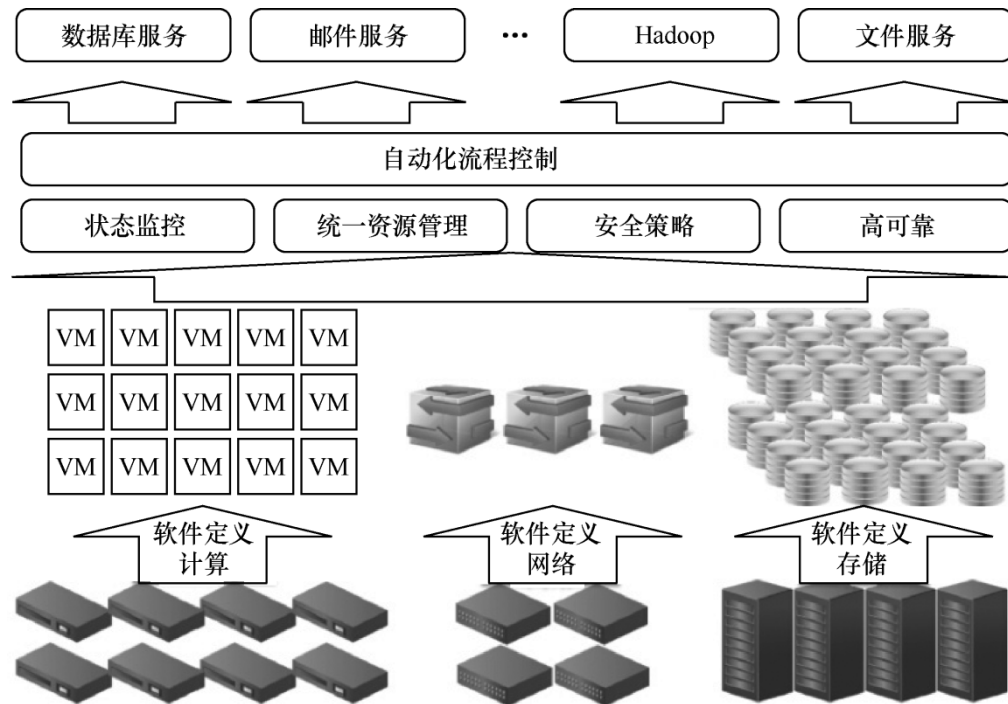


图3. 31 软件定义数据中心的分层模型

# 1. 层次细分

传统的数据中心中系统软件和应用软件处于硬件之上。但是在软件定义数据心里，硬件的能力需要被抽象成为能够统一调度管理的资源池，而且计算、存储和网络资源的抽象方式各不相同。

- (1) 软件定义计算
- (2) 软件定义存储
- (3) 软件定义网络

## 2. 接口与标准

|               | 成熟度                                                           | 开放性                                                             |
|---------------|---------------------------------------------------------------|-----------------------------------------------------------------|
| VMware        | 成熟的API，涵盖了资源管理、状态监控、性能分析等各方面。API相对稳定，并有清晰的发展路线图               | 比较开放的接口标准，有成熟的开发社区和生态系统，是企业级厂商选择兼容的首选                           |
| OpenStack     | 软件定义计算的API相对成熟和稳定，但是存储、网络、监控、自动化管理等部分API比较初级，不适用于生产环境，需要进一步加强 | 完全开放的接口标准并且计算与存储服务能够兼容AWS的API                                   |
| System Center | 成熟的API                                                        | 不够开放的标准，有开发社区做支撑                                                |
| CloudStack    | 比较成熟的 API，比较新的功能如自动化管理和网络管理由开源社区实现                            | 原本作为单独的产品发布，接口与开发人员不完全开放。后转为由开源社区支持，大部分API均已开放。计算与存储服务兼容AWS的API |

## 3.4.5 软件定义数据中心的发展

1. 需求推动，先行者不断
2. 新技术不断涌现，发展迅速
3. 发展空间巨大，标准建立中

## 3.5 实践： OpenStack

3.5.1 初识OpenStack

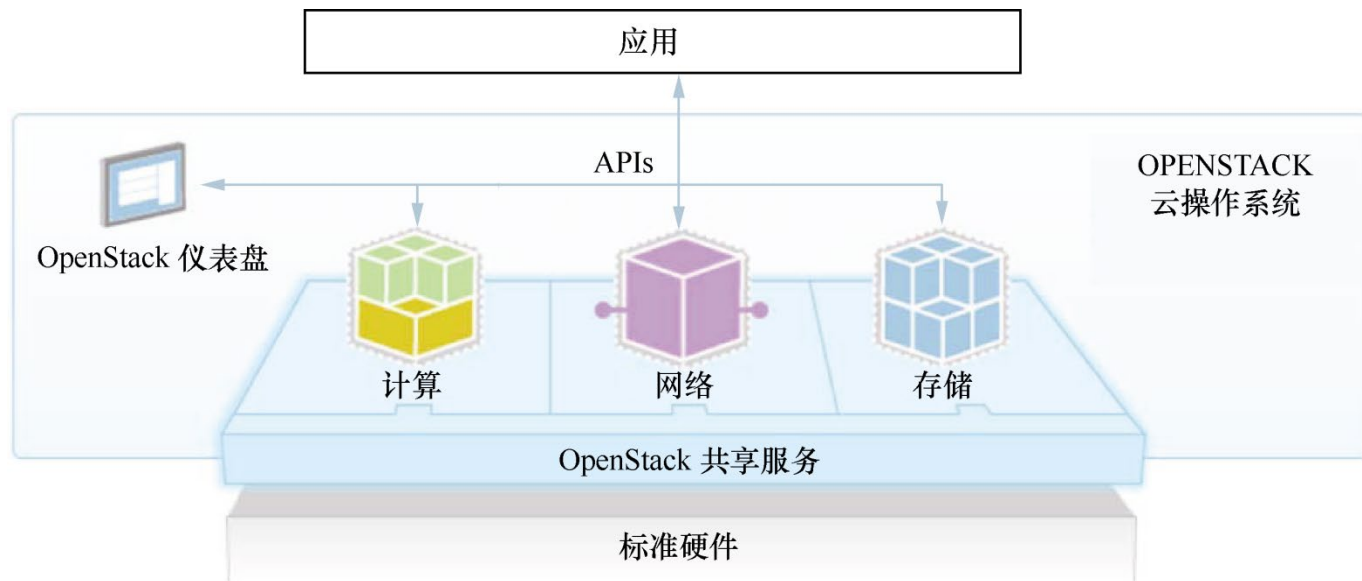
3.5.2 OpenStack组件介绍

3.5.3 体验使用OpenStack

## 3.5.1 初识OpenStack

- OpenStack提供了一个通用的平台来控制云计算里面的计算（服务器）、存储和网络，甚至应用资源。
- OpenStack可以通过基于Web的界面、命令行工具（CLI）和应用程序接口（API）来进行管理。
- 这个管理平台不仅能管理这些资源，而且它不需要用户去选择特定硬件或者软件厂商。厂商特定组件可以方便地被替换成通用组件，OpenStack为IT业界各类从业人员创造了价值。

# 图3.32 Openstack结构

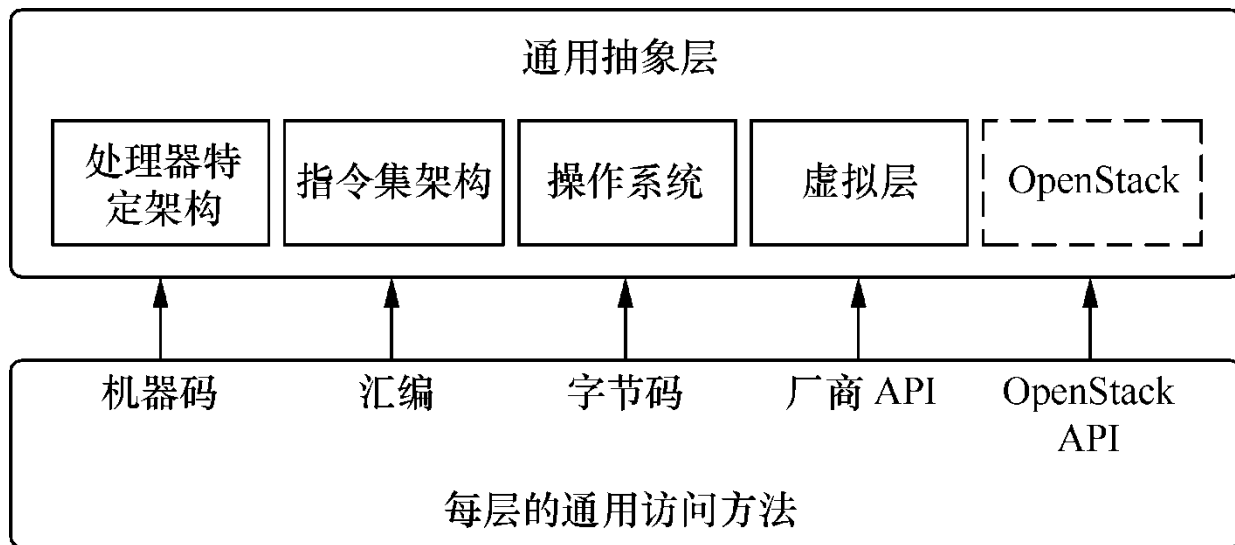


## 3.5.1 初识OpenStack

在OpenStack云平台上，用户可以做到以下几个方面。

- 充分利用物理服务器、虚拟服务器、网络和存储系统资源；
- 通过租户、配额和用户角色高效管理云资源；
- 提供一个对底层实现透明的通用的资源控制接口。

# 图3.33 计算抽象的层次



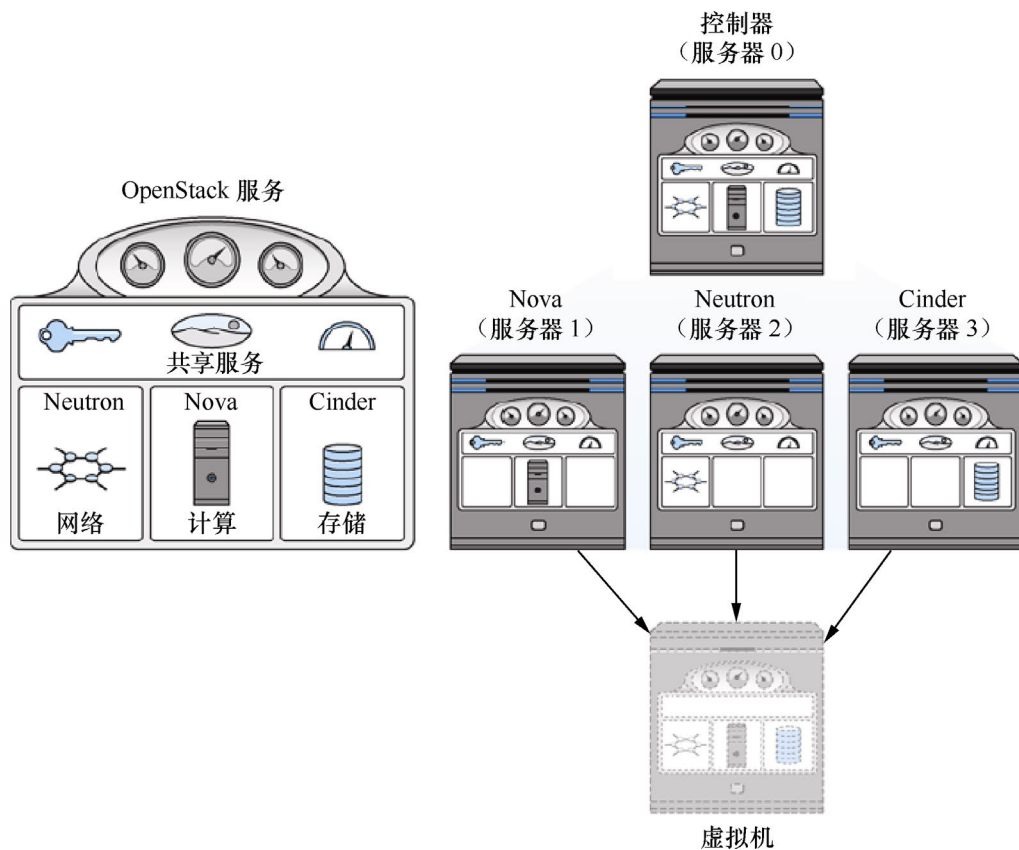
## 3.5.2 OpenStack组件介绍

| 项目                   | 代码名称       | 描述                                   |
|----------------------|------------|--------------------------------------|
| 计算（Compute）          | Nova       | 管理虚拟机资源，包括CPU、内存、磁盘和网络接口             |
| 网络（Networking）       | Neutron    | 提供虚拟机网络接口资源，包括IP寻址、路由和软件定义网络（SDN）    |
| 对象存储（Object Storage） | Swift      | 提供可通过RESTful API访问的对象存储              |
| 块存储（Block Storage）   | Cinder     | 为虚拟机提供块（传统磁盘）存储                      |
| 身份认证服务（Identity）     | Keystone   | 为OpenStack组件提供基于角色的访问控制（RBAC），提供授权服务 |
| 镜像服务（Image Service）  | Glance     | 管理虚拟机磁盘镜像，为虚拟机和快照（备份）服务提供镜像          |
| 仪表盘（Dashboard）       | Horizon    | 为OpenStack提供基于Web的图形界面               |
| 计量服务（Telemetry）      | Ceilometer | 集中为OpenStack各个组件收集计量和监控数据            |
| 编排服务（Orchestration）  | Heat       | 为OpenStack环境提供基于模板的云应用编排服务           |

## 3.5.3 体验使用OpenStack

用户可以通过DevStack与一个小规模（更大规模部署的代表）的OpenStack交互，不需要深入了解OpenStack，也不需要大量硬件，就可以在一个小规模范围内通过DevStack来体验OpenStack。可以快速部署这些组件，来评估在生产用例中的使用。DevStack可以帮助用户在一个单服务器环境中部署与大规模多服务器环境中一样的OpenStack组件。

# 图3.34 OpenStack的相关组件



# 小结



summary

- 云计算的本质
- 云计算的架构
- 云栈和云体
- 软件定义的数据中心
- 实践：OpenStack

# 课内复习

- 1. 云计算中的工作负载有哪几种模式？它们的特征是什么？
- 2. 如何避免云计算资源“超配”带来的问题？
- 3. 如何理解“云栈”和“云体”的概念？
- 4. 什么是软件定义的数据中心？它的特点是什么？

# 课外思考

- 1. 云计算的架构是如何演化的？
- 2. 如何理解“软件定义一切”的说法？

# 动手实践

OpenStack是一个旨在为公共及私有云的建设与管理提供软件的开源项目。OpenStack支持几乎所有类型的云环境，项目目标是提供实施简单、可大规模扩展、丰富、标准统一的云计算管理平台。

- 任务：通过OpenStack的官方网站全面了解OpenStack作为一个开源项目的情况。
- 任务：通过DevStack工具来安装并体验OpenStack。

# Thanks!

